APE 4 - Sistema de reservas de vuelos

Guía de arquitectura, instalación, IVA, comprobante XML y despliegue

1. Objetivo

Aplicación SOA para consultar vuelos y gestionar reservas. Flutter consume servicios REST en Flask; Flask valida la identidad con JWT y accede a Supabase/PostgreSQL. El cliente móvil nunca se conecta directamente a la base de datos.

2. Arquitectura SOA

Cliente Flutter	API REST Flask	Supabase / PostgreSQL
Pantallas, modelos y solicitudes HTTPS/JSON	Registro, autenticación, vuelos, reservas y XML	usuarios, tokens, vuelos y reservas

La clave service_role permanece solo en el backend. Las reservas actualizan inventario y guardan subtotal, IVA y total en una operación transaccional.

3. Flujo y modelo

Paso	Comportamiento
1	Registro e inicio de sesión; Flask entrega un JWT con expiración.
2	Flutter consulta vuelos y filtra por origen, destino o fecha.
3	PostgreSQL descuenta asientos e inserta la reserva sin sobreventa.
4	Se calcula subtotal, IVA del 15 % redondeado a centavos y total.
5	El propietario consulta reservas y su comprobante XML autenticado.
6	Al cancelar, se cambia el estado y se restauran los asientos.

Tabla	Datos principales
usuarios	id, nombre, correo único y hash de contraseña
tokens	id del token, usuario y expiración
vuelos	origen, destino, fecha, horarios, precio y asientos
reservas	usuario, vuelo, cantidad, subtotal, IVA, porcentaje, total y estado

4. Supabase y cálculo

Base nueva: ejecuta `database/schema.sql` y luego `database/semilla.sql` en SQL Editor.

Base existente: ejecuta `database/migrations/001_iva_reservas.sql` antes de actualizar el backend. Es idempotente: conserva los totales históricos, los registra como subtotal con IVA cero y no aplica impuestos retroactivos. Render no ejecuta la migración.

Importe	Regla
Subtotal	Precio del vuelo × cantidad
IVA	Redondear subtotal × 0.15 a dos decimales
Total	Subtotal + IVA

El XML es un comprobante académico de reserva; no es factura fiscal ni documento autorizado por el SRI.

5. Ejecutar localmente

Desde la raíz del proyecto, en PowerShell:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r backend/requirements.txt
Copy-Item backend/.env.example backend/.env
```

Completa backend/.env con SUPABASE_URL, SUPABASE_SERVICE_KEY y JWT_SECRET aleatorio de al menos 32 bytes. Inicia Flask:

```
python -m backend.app
```

La API queda en http://localhost:5000. No hace falta Docker.

En otra terminal:

```
cd aplicacion_flutter
flutter pub get
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:5000
```

10.0.2.2 apunta al anfitrión desde el emulador. HTTP solo se habilita en debug; release requiere HTTPS.

6. Endpoints REST

Método y ruta	Uso
POST /api/usuarios/registro	Crear cuenta
POST /api/auth/login	Iniciar sesión y recibir JWT
GET /api/vuelos	Buscar y filtrar vuelos
POST /api/reservas	Reservar y calcular importes
GET /api/reservas	Listar reservas propias
GET /api/reservas/<id>	Consultar reserva propia
GET /api/reservas/<id>/comprobante.xml	Consultar XML propio
DELETE /api/reservas/<id>	Cancelar reserva propia

7. Despliegue en Render

El Blueprint `render.yaml` crea un servicio web Python separado. No reutilices AD-2026 ni otros servicios ajenos.

Paso	Acción
1	Crea un Blueprint y selecciona el repositorio AHJimmy06/adape4 que contiene render.yaml.
2	Aplica la migración Supabase si la base ya existía; Render no modifica la base.
3	Define SUPABASE_URL y SUPABASE_SERVICE_KEY como secretos; el Blueprint genera JWT_SECRET.
4	Espera el despliegue y verifica https://<servicio>.onrender.com/api/vuelos.
5	Configura Flutter release con --dart-define=API_BASE_URL=https://<servicio>.onrender.com.

Gunicorn inicia en 0.0.0.0:$PORT y /api/vuelos sirve de health check. No guardes secretos en render.yaml, Flutter ni Git. Referencias: Blueprints y variables secretas.

8. Ejemplo XML

```
<?xml version="1.0" encoding="utf-8"?>
<comprobante tipo="reserva" fiscal="false">
  <numero>reserva-id</numero>
  <fecha>2026-11-09T10:00:00+00:00</fecha>
  <cliente>Nombre del pasajero</cliente>
  <pasajeros>1</pasajeros>
  <subtotal>50.00</subtotal>
  <iva porcentaje="15.00">7.50</iva>
  <total>57.50</total>
</comprobante>
```

GET /api/reservas/<id>/comprobante.xml requiere Authorization: Bearer <token>. Flask valida la propiedad y escapa el XML.

9. Seguridad y pruebas

Las contraseñas se guardan con scrypt; cada ruta privada requiere JWT activo y acceso a reservas propias. Flask usa service_role y RLS bloquea el acceso directo. El comprobante es demostrativo, no reemplaza facturación fiscal.

```
python -m pytest backend/tests -q
cd aplicacion_flutter
flutter analyze
flutter test
```

Las pruebas usan un repositorio falso y no requieren secretos. La conexión real a Supabase requiere credenciales del titular. Docker no es necesario.